P R O D U C T R E Q U I R E M E N T S D O C U M E N T

Archimedes

AI-Assisted Architecture Intelligence Platform

Composable Codebase Intelligence for Software Modernization

Version 1.0 • Draft

February 2026

Adam • Delos Living

Table of Contents

Table of Contents 2
1. Executive Summary 4
  1.1 Problem Statement 4
  1.2 Proposed Solution 4
  1.3 Core Value Proposition 4
  1.4 Key Differentiator 4
2. Vision & Strategic Context 6
  2.1 Product Vision 6
  2.2 Phased Strategy 6
  2.3 Business Model 6
  2.4 Target Users 6
3. Three-Tier Intelligence Architecture 7
  3.1 Tier 1: Deterministic Code Intelligence 7
    3.1.1 Structural Pattern Matching (ast-grep) 7
    3.1.2 Architecture Flow Extraction (Rivière) 7
    3.1.3 Infrastructure-as-Code as Rosetta Stone 7
  3.2 Tier 2: Semantic Retrieval 7
    3.2.1 Code Chunking (code-chunk) 8
    3.2.2 Multi-Vector Semantic Search (ColGREP / next-plaid) 8
    3.2.3 Hybrid Local Search (qmd) 8
  3.3 Tier 3: Agentic Search & Navigation 8
    3.3.1 Structural Code Intelligence (osgrep) 8
    3.3.2 Live Symbolic Navigation (Serena) 8
    3.3.3 Deep Reasoning Search (rlmgrep) 9
    3.3.4 Multi-Hop Research (ChunkHound) 9
4. Tag Store: The Persistent Knowledge Model 10
  4.1 Design Principles 10
  4.2 Tag Schema 10
  4.3 Tag Lifecycle & Feedback Loop 11
  4.4 Tag Weighting 11
5. Skill Architecture 13
  5.1 Skill Layering 13
    5.1.1 Layer 1: Atomic Skills 13
    5.1.2 Layer 2: Workflow Skills 13

5.1.3 Layer 3: Autonomous Skills 14
5.2 Skill Design Principles 14
6. Cross-Repository Coordination 16
6.1 Multi-Repo Session Model 16
6.2 Confidence Gradient by Source 16
7. Tool Inventory & Integration Requirements 18
7.1 Integration Requirements 18
8. Output Deliverables 20
8.1 Architecture Map 20
8.2 Service Profiles 20
8.3 Technical Debt Assessment 20
8.4 Modernization Roadmap 20
8.5 Tag Store Export 20
9. Success Metrics 21
9.1 v1 Metrics (Consulting Accelerator) 21
9.2 v2 Metrics (Platform) 21
10. Risks & Open Questions 22
10.1 Technical Risks 22
10.2 Open Questions 22
11. Implementation Roadmap 24
11.1 Phase 1: Foundation (Weeks 1–4) 24
11.2 Phase 2: Semantic Layer (Weeks 5–8) 24
11.3 Phase 3: Agentic Layer (Weeks 9–12) 24
11.4 Phase 4: Autonomy (Weeks 13–16) 24
Appendix A: Tool Reference Links 24

Note

This Table of Contents will populate when you open the document in Word and press Ctrl+A then F9, or right-click the TOC and select “Update Field.”

1. Executive Summary

1.1 Problem Statement

Software architecture modernization is one of the highest-value, highest-complexity consulting engagements in the industry. Organizations with large, distributed codebases—often spanning hundreds of repositories, multiple languages, and years of accumulated technical debt—struggle to understand what they actually have, let alone chart a path forward. Existing approaches are either purely manual (expensive, slow, inconsistent) or rely on single-dimensional tooling that captures code structure but misses architectural intent, operational flows, and the relationship between infrastructure and implementation.

The core challenge is that architectural knowledge is scattered across code, infrastructure-as-code, documentation, commit history, and tribal knowledge. No single tool provides the layered intelligence needed to synthesize these sources into actionable modernization guidance. Consultants spend the majority of their engagement time on discovery and mapping rather than analysis and recommendation.

1.2 Proposed Solution

Archimedes is an AI-assisted architecture intelligence platform that provides layered codebase understanding through composable, autonomous tooling. It combines deterministic code analysis, semantic retrieval, and agentic search into a unified system that progressively builds architectural knowledge through a persistent tag store. The platform operates as a family of composable agent skills that can be used interactively or orchestrated autonomously to conduct comprehensive architecture assessments.

1.3 Core Value Proposition

For the consultant/architect: Reduce discovery and mapping time from weeks to hours. Replace manual code archaeology with systematic, repeatable, evidence-based analysis. Build a cumulative understanding of a codebase that persists across sessions and deepens over time.

For the client organization: Receive modernization assessments grounded in concrete evidence rather than sampling and intuition. Get actionable, prioritized recommendations tied to specific code, infrastructure, and documentation. Retain the architectural knowledge map as a living artifact beyond the engagement.

1.4 Key Differentiator

Three-Tier Intelligence with Progressive Knowledge Accumulation

Unlike tools that provide a single analytical lens, Archimedes layers three tiers of intelligence—deterministic rules, semantic retrieval, and agentic search—connected by a persistent tag store that accumulates findings with calibrated confidence weights.

Discoveries from autonomous exploration feed back into the deterministic layer, creating a system that gets smarter with each query rather than starting fresh.

2. Vision & Strategic Context

2.1 Product Vision

Archimedes transforms software architecture modernization from an artisanal, expertise-bottlenecked practice into a systematic, tool-augmented discipline. The long-term vision is a platform where an architect can point the system at an organization's codebase, infrastructure, and documentation—and receive a comprehensive architectural map with identified modernization opportunities, risk assessments, and prioritized migration paths.

2.2 Phased Strategy

v1 — Consulting Accelerator (Target: 2026). A composable skill-based toolkit that augments an expert consultant's workflow. The consultant drives the investigation, using Archimedes skills through Claude Code or similar agent interfaces. Value delivery: 5–10x faster discovery phase, evidence-backed assessments, persistent knowledge accumulation across sessions.

v2 — Unified Platform (Target: 2027). Extract the best primitives from the v1 toolchain into a cohesive, self-contained platform. Replace shell-outs to external CLIs with integrated implementations. Add a web-based frontend for non-CLI users. Enable SaaS delivery for self-service architecture assessments.

2.3 Business Model

The v1 product is a consulting accelerator—it magnifies the value of expert architecture consulting engagements. Revenue comes from consulting services delivered faster and at higher quality. The v2 product enables either a SaaS platform (recurring revenue from organizations running self-service assessments) or a productized consulting offering (standardized assessments at lower price points with higher volume). The v1 toolchain also has value as developer tooling exposed through agent skills and MCP integrations.

2.4 Target Users

Persona	Context	Primary Need
Architecture Consultant	Engaged by client to assess and modernize their systems	Rapid, evidence-based codebase understanding across dozens to hundreds of repos
Staff/Principal Engineer	Internal champion for modernization within their organization	Systematic mapping of technical debt, dependencies, and migration risks
Engineering Director	Sponsor of modernization initiative	Data-driven prioritization of modernization investments with clear ROI framing

3. Three-Tier Intelligence Architecture

The core architectural insight of Archimedes is that no single analytical approach provides sufficient understanding of a complex software system. Deterministic tools find what they're told to look for. Semantic search finds what's similar to what you ask about. Agentic search follows threads of investigation to find what you didn't know to ask about. All three are necessary, and their findings must be unified in a shared knowledge model.

3.1 Tier 1: Deterministic Code Intelligence

Deterministic tools produce high-confidence, zero-false-positive findings through structural pattern matching and schema-driven extraction. These findings form the bedrock of the tag store—they are facts, not interpretations.

3.1.1 Structural Pattern Matching (ast-grep)

ast-grep provides structural search and lint over abstract syntax trees. Unlike regex-based search, it understands code structure—matching on AST node types, wildcards for expressions, and nested patterns. For architecture modernization, this enables detection of anti-patterns (e.g., direct database calls outside repository layers), technology usage patterns (e.g., all Redis client instantiations), API surface enumeration, and dependency injection patterns.

Key capability: Zero false positives. When ast-grep matches a pattern, that pattern exists in the code. This makes its findings suitable for direct promotion to human-weight tags without manual validation.

3.1.2 Architecture Flow Extraction (Rivière)

Rivière, from Nick Tune's living-architecture project, captures operational flows as JSON schemas describing how operations traverse system boundaries: UI → API → domain logic → events. This is not a technical dependency graph—it's a model of how the system actually works from a user's perspective. Rivière provides a TypeScript SDK for programmatic schema construction and a CLI designed for AI-assisted extraction. Éclair provides the visualization layer with domain filtering, flow tracing, and event pub/sub tracking.

Key capability: Rivière schemas become the target representation for Archimedes' architectural understanding. The goal of a full assessment is to produce populated Rivière schemas that map every significant operation flow, annotated with tags from all three tiers.

3.1.3 Infrastructure-as-Code as Rosetta Stone

IaC repositories (Terraform, CDK, CloudFormation, Kubernetes manifests) are machine-readable system topology declarations—the one place where architectural truth is enforced, because infrastructure won't deploy if the declaration is wrong. Archimedes uses IaC as a bootstrap layer: parse with ast-grep patterns, extract the dependency graph (what talks to what, over which protocols), and use this as the scaffold for Rivière flow schemas. For systems without clean IaC, the confidence gradient degrades through Docker Compose → Kubernetes manifests → service mesh configs → code-level dependency tracing.

3.2 Tier 2: Semantic Retrieval

Semantic retrieval discovers relationships and patterns that deterministic tools cannot see—conceptual similarity, intent alignment, and latent structure in code and documentation. These findings carry machine-weight confidence and serve as candidates for investigation or promotion.

3.2.1 Code Chunking (code-chunk)

Code-chunk from Supermemory implements the cAST algorithm (Carnegie Mellon, arXiv:2506.15655) for AST-aware code chunking. Its key differentiator is contextualized text output: each chunk includes scope chains, import dependencies, and sibling context prepended as metadata. This enrichment gives embedding models the semantic signals needed to distinguish between identically-named methods in different classes and understand a function's role within its module. The library supports TypeScript, JavaScript, Python, Rust, Go, and Java with configurable context modes and overlap.

3.2.2 Multi-Vector Semantic Search (ColGREP / next-plaid)

ColGREP from LightOn implements ColBERT-style multi-vector search where each document is represented by approximately 300 embeddings of dimension 128 rather than a single vector. MaxSim queries find the best token-level matches between query and document, providing significantly finer-grained semantic matching than single-vector approaches. The system ships as a single Rust binary with built-in ONNX Runtime, memory-mapped indices for low RAM usage, and auto-updating file watch support.

3.2.3 Hybrid Local Search (qmd)

qmd from Shopify provides fully local hybrid search combining BM25 full-text search, vector semantic search, and LLM-powered reranking using local GGUF models. Its query pipeline expands the original query via a fine-tuned LLM, runs parallel BM25 and vector retrieval, fuses results with Reciprocal Rank Fusion, and applies position-aware blending between RRF scores and LLM reranker judgments. Context tree annotations provide hierarchical metadata with results. qmd is particularly valuable for documentation and knowledge base search where lexical matching complements semantic similarity.

3.3 Tier 3: Agentic Search & Navigation

Agentic tools follow threads of investigation, tracing dependencies, classifying roles, and synthesizing findings across sources. Their discoveries feed back into the tag store and, after validation, can be promoted to deterministic-weight tags.

3.3.1 Structural Code Intelligence (osgrep)

osgrep provides semantic code search with structural understanding: split querying (code vs. documentation), ColBERT reranking, structural boosting (functions and classes scored higher than test files), and role classification that distinguishes ORCHESTRATION functions (high complexity, many outgoing calls) from DEFINITION functions (types, interfaces, data structures). Its call-chain analysis traces upstream and downstream dependencies. A daemon mode with file watching provides sub-50ms responses for interactive use.

3.3.2 Live Symbolic Navigation (Serena)

Serena wraps Language Server Protocol servers as MCP tools, providing find_symbol, find_references, type_hierarchy, and rename_symbol capabilities with zero indexing cost. It offers precise, live code navigation that complements semantic search—when you know what you're looking for, LSP gives you exact answers. The JetBrains plugin backend enables polyglot support with full dependency resolution.

3.3.3 Deep Reasoning Search (rlmgrep)

rlmgrep uses DSPy-powered reasoning language models to search not just for lexical or semantic similarity but for conceptual answers to natural language questions. It processes PDFs, Office documents, images (via vision models), and audio (via transcription) alongside code. For architecture modernization, this enables mining ADRs, Confluence exports, design documents, and onboarding guides for architectural intent that doesn't exist in code.

3.3.4 Multi-Hop Research (ChunkHound)

ChunkHound's research layer performs BFS-based multi-hop exploration through the semantic graph, expanding queries into multiple entry points and following symbol references with adaptive token budgets. With approximately 30 language support and stress-testing on the Kubernetes codebase (4.8M lines, 23K files), it handles large-scale codebase exploration that would overwhelm simpler search tools.

4. Tag Store: The Persistent Knowledge Model

The tag store is the load-bearing primitive of Archimedes. Every tool in every tier reads from and writes to the tag store. It is the mechanism by which the system accumulates understanding over time rather than starting fresh with each query, and the mechanism by which autonomous discoveries feed back into the deterministic layer.

4.1 Design Principles

- Tags are the universal unit of architectural knowledge
- Every tag has a source tool, confidence weight, and provenance chain
- Tags attach to a hierarchy of targets: repository → file → symbol/range, plus cross-cutting concepts (flows, domains, bounded contexts)
- The tag lifecycle—candidate → validated → promoted—is the formalized feedback loop from Tier 3 back to Tier 1
- Storage is local-first (SQLite), portable as a session artifact, and queryable by all tools regardless of implementation language

4.2 Tag Schema

Field	Type	Description
id	UUID	Unique tag identifier
target_type	Enum	REPO | FILE | SYMBOL | RANGE | FLOW | DOMAIN | RESOURCE
target_ref	String	Identifier for the target (file path, symbol name, flow ID, IaC resource ARN)
target_repo	String	Repository identifier (git remote URL or local path)
kind	String	Tag category: PATTERN | DEPENDENCY | ROLE | DEBT | FLOW | BOUNDARY | CAPABILITY | RISK
value	JSON	Structured payload specific to tag kind
confidence	Float	0.0–1.0 confidence score
weight_class	Enum	HUMAN | MACHINE | PROMOTED
source_tool	String	Tool that produced this tag (ast-grep, osgrep, rlmgrep, etc.)
source_query	String	The query or pattern that generated this finding
source_evidence	Text	Raw evidence (code snippet, document excerpt, search result)
status	Enum	CANDIDATE | VALIDATED | PROMOTED | REJECTED | STALE
parent_tag_id	UUID?	Link to parent tag for derived findings
related_tags	UUID[]	Links to related tags across targets
session_id	UUID	Session that produced this tag
created_at	Timestamp	Creation time

Field	Type	Description
updated_at	Timestamp	Last modification time
validated_by	String?	Human or agent that validated this tag

4.3 Tag Lifecycle & Feedback Loop

The tag lifecycle formalizes the progressive knowledge accumulation that distinguishes Archimedes from single-query tools:

1. **CANDIDATE** — A tool produces a finding. Machine-weight tags from semantic or agentic tools start here. Deterministic findings from ast-grep may skip directly to VALIDATED.
2. **VALIDATED** — A finding has been corroborated by a second source (another tool, a human review, or cross-referencing with existing tags). Confidence score increases.
3. **PROMOTED** — A machine-weight finding has been reviewed and accepted as high-confidence architectural knowledge. It now carries human-weight authority and can be used by deterministic rules in future queries.
4. **REJECTED** — A finding was investigated and determined to be incorrect or irrelevant. Retained for provenance but excluded from active queries.
5. **STALE** — A finding's source evidence has changed (file modified, dependency updated). Flagged for re-evaluation.

The Feedback Loop

When the autonomous agent (Layer 3 skill) discovers that Service A calls Service B through an undocumented queue, it writes a CANDIDATE tag with machine weight. If the consultant validates this finding, the tag is PROMOTED to human weight. Future deterministic scans now include this queue relationship as a known fact, enabling ast-grep patterns to verify the queue configuration and Rivière schemas to include this flow. The system's knowledge compounds.

4.4 Tag Weighting

Tags carry a weight class that determines their authority in downstream reasoning:

Weight Class	Source	Authority	Use in Reasoning
HUMAN	Manual entry, promoted machine tags, validated documentation	Highest — treated as architectural fact	Deterministic rules, Rivière schema population, constraint checking
PROMOTED	Machine findings validated by human or corroborating evidence	High — trusted but flagged as derived	Flow tracing, dependency mapping, modernization recommendations

Weight Class	Source	Authority	Use in Reasoning
MACHINE	Autonomous tool findings, semantic search results	Moderate — candidate for investigation	Hypothesis generation, guided exploration, risk flagging

5. Skill Architecture

Archimedes operates as a family of agent skills rather than a monolithic application. Skills are the primary interface—they encode both tool invocation knowledge and consulting methodology. The agent (Claude Code or similar) orchestrates skills based on the task, while the skills constrain and guide the agent's decisions.

5.1 Skill Layering

5.1.1 Layer 1: Atomic Skills

Single-purpose skills that wrap individual tools. Each skill knows its tool's CLI interface, output contract, and the kinds of tags it produces and consumes.

Skill	Wraps	Produces Tags	Consumes Tags
arch-structure	ast-grep	PATTERN, DEPENDENCY (human-weight)	None (bootstrap)
arch-flows	Rivière CLI	FLOW, BOUNDARY (human-weight)	PATTERN, DEPENDENCY
arch-search	code-chunk + ColGREP	DEPENDENCY, CAPABILITY (machine-weight)	PATTERN (for guided search)
arch-docs	qmd + rlmgrep	FLOW, DEBT, RISK (machine-weight)	FLOW (for targeted doc mining)
arch-navigate	Serena LSP	DEPENDENCY, ROLE (machine-weight)	SYMBOL targets from other tags
arch-observe	osgrep	ROLE, DEPENDENCY, DEBT (machine-weight)	PATTERN (for structural boosting)
arch-tags	Tag store (SQLite)	N/A (meta-skill)	All tags (read/write/query)

5.1.2 Layer 2: Workflow Skills

Multi-step skills that compose atomic skills into repeatable investigative workflows. These encode the consulting methodology—the sequencing and decision logic that an expert architect applies.

Skill	Composes	Purpose
arch-map-service	arch-structure → arch-search → arch-observe → arch-tags	Given a repository, produce a comprehensive service profile: API surface, dependencies, role classifications, technology patterns, and tagged findings

Skill	Composes	Purpose
arch-trace-flow	arch-navigate → arch-search → arch-docs → arch-flows	Given an operation name, trace it across repositories and produce a populated Rivière flow schema
arch-assess-debt	arch-structure → arch-observe → arch-docs → arch-tags	Run structural and role analysis across repositories, classify technical debt concentrations, cross-reference with documentation
arch-map-infra	arch-structure (IaC) → arch-search → arch-tags	Parse IaC repos to extract system topology, map resources to implementing services, produce infrastructure dependency graph
arch-audit-consistency	arch-tags → arch-structure → arch-navigate	Compare tag store assertions against current code reality, flag stale tags, identify drift between documented and actual architecture

5.1.3 Layer 3: Autonomous Skills

Meta-skills that take a high-level goal and have permission to invoke workflow skills, interpret results, decide what to investigate next, and accumulate tags over extended sessions.

Skill	Capability	Guardrails
arch-modernize	Full-scope modernization assessment: map services, trace flows, assess debt, identify migration candidates, produce prioritized recommendations	Operates within defined repo scope; writes only CANDIDATE and MACHINE-weight tags; requires human review for PROMOTED status; reports confidence levels for all recommendations
arch-investigate	Open-ended architectural investigation: given a question or hypothesis, autonomously explore the codebase to find evidence	Time-bounded sessions; tag everything found; surface conflicting evidence; explicit uncertainty reporting

5.2 Skill Design Principles

- Skills are CLI-wrapping agent prompts, not MCP servers. The agent shells out to tools and interprets results.
- Every skill reads from and writes to the tag store. The tag store is the shared memory that enables composition.
- Skills declare their tag production and consumption contracts. This enables the agent to reason about sequencing.
- Workflow skills encode methodology, not just tool sequencing. They include decision points: “if the structure scan reveals more than N microservices, switch to service-group analysis rather than individual mapping.”

- Autonomous skills have explicit guardrails: scope boundaries, time limits, confidence thresholds for acting on findings, and mandatory uncertainty reporting.

6. Cross-Repository Coordination

Architecture modernization fundamentally requires understanding how services interact across repository boundaries. Most code intelligence tools index individual repositories well but cannot answer questions like “which services would be affected if we change the user event schema?” Archimedes addresses this through the tag store’s cross-cutting concepts and multi-repo indexing.

6.1 Multi-Repo Session Model

An Archimedes session targets a defined scope of repositories. The session initialization process indexes each repository individually (using each tool’s per-repo indexing capability), then builds cross-repo connections through the tag store:

1. Parse IaC repositories to extract the system topology graph (the authoritative map of what talks to what)
2. Index each service repository with semantic search (ColGREP, osgrep) and structural analysis (ast-grep)
3. Use IaC topology as a scaffold to discover cross-repo dependencies: if IaC says Service A sends to Queue X and Service B reads from Queue X, search both repos for queue client code and tag the relationship
4. Mine documentation (qmd, rlmgrep) for architectural decisions, known limitations, and migration plans that span services
5. Populate Rivière flow schemas that trace operations across service boundaries, linking tags from individual repos into coherent flows

6.2 Confidence Gradient by Source

Cross-repo findings carry different confidence levels depending on their provenance:

Source	Confidence	Rationale
IaC declarations (Terraform, CDK)	0.95	Machine-enforced truth; infrastructure won’t deploy if wrong
Docker Compose / K8s manifests	0.85	Deployment truth, but may not reflect all runtime communication
Service mesh / API gateway configs	0.80	Operational truth, but configs may be stale or overridden
Code-level dependency tracing (imports, SDK calls)	0.70	Implementation truth, but may include dead code or conditional paths
Semantic search similarity	0.50	Suggestive but requires validation; conceptual match, not proven dependency
Documentation / ADR references	0.60	Intent truth, but may be outdated or aspirational rather than implemented

7. Tool Inventory & Integration Requirements

Archimedes v1 composes existing open-source and commercial tools rather than building from scratch. Each tool has been evaluated for its integration surface, maturity, and unique contribution to the three-tier intelligence model.

Tool	Language	Interface	Tier	Unique Contribution
ast-grep	Rust	CLI	1	Structural pattern matching with zero false positives
Rivière / Éclair	TypeScript	CLI + SDK	1	Operation flow schemas capturing how systems actually work
code-chunk	TypeScript	npm library	2	Best-in-class contextualized code chunks for embedding
ColGREP (next-plaid)	Rust	CLI + REST API	2	Multi-vector semantic search with token-level matching
qmd	TypeScript	CLI + MCP + HTTP	2	Fully local hybrid search with LLM reranking
osgrep	Python	CLI + daemon + hooks	3	Role classification + call-chain analysis
Serena	Python	MCP	3	Live LSP symbolic navigation
rlmgrep	Python	CLI	3	LLM reasoning search across docs, PDFs, images, audio
ChunkHound	Python	CLI + MCP	2/3	30-language coverage + multi-hop research

7.1 Integration Requirements

Each tool must satisfy the following integration contract to participate in the Archimedes skill system:

- **CLI invocation:** Tool must be invocable via command-line with predictable, parseable output (JSON preferred, structured text acceptable)
- **Per-repo indexing:** Tool must support indexing individual repositories with a stable index identifier (git remote URL preferred)
- **Incremental updates:** Tool should support re-indexing modified files without full reindex (required for daemon/watch mode)
- **Output contract:** Tool output must be stable and documented, suitable for agent parsing and tag store ingestion
- **Error isolation:** Failures in one tool must not cascade to other tools or corrupt the tag store)

8. Output Deliverables

An Archimedes assessment produces the following artifacts, each grounded in the tag store:

8.1 Architecture Map

A populated set of Rivière flow schemas visualized through Éclair, showing every significant operation flow with annotated service boundaries, communication protocols, and data flows. Each element is linked to supporting tags with confidence scores.

8.2 Service Profiles

Per-service summaries including: API surface, inbound and outbound dependencies, technology stack, role classification (orchestrator vs. domain service vs. utility), technical debt concentration, and modernization readiness score. Generated by the arch-map-service workflow skill.

8.3 Technical Debt Assessment

Classified and prioritized technical debt findings with evidence links to specific code, infrastructure, and documentation. Categories include: coupling hotspots, abstraction violations, infrastructure drift, documentation gaps, and migration blockers. Generated by the arch-assess-debt workflow skill.

8.4 Modernization Roadmap

Prioritized recommendations with dependency ordering, risk assessment, and effort estimation. Each recommendation is tied to specific tags, enabling traceability from recommendation back to evidence. Generated by the arch-modernize autonomous skill with human review.

8.5 Tag Store Export

The complete tag store exported as a portable SQLite database. This is the client's lasting artifact—a queryable knowledge base of their architecture that can be updated in subsequent engagements or maintained internally.

9. Success Metrics

9.1 v1 Metrics (Consulting Accelerator)

Metric	Target	Measurement
Discovery phase duration	70% reduction vs. manual baseline	Time from engagement start to first architecture map delivery
Tag accuracy (promoted)	≥90% validated on human review	Percentage of MACHINE tags that survive promotion review
Cross-repo flow coverage	≥80% of IaC-declared dependencies mapped	Rivière flows populated vs. IaC topology graph
Recommendation traceability	100% of recommendations linked to tags	Every modernization recommendation has evidence chain
Session accumulation	Measurable improvement across sessions	Tag store quality and coverage metrics improve with each session

9.2 v2 Metrics (Platform)

Metric	Target	Measurement
Self-service assessment completion	Non-expert user can produce useful architecture map	Task completion rate without consultant intervention
Time to first insight	<30 minutes from repo connection to first findings	Elapsed time from session start to actionable tags
Tool integration latency	<5s for atomic skill, <5min for workflow skill	End-to-end skill execution time

10. Risks & Open Questions

10.1 Technical Risks

Risk	Severity	Mitigation
Tool ecosystem fragility: depending on 9+ open-source tools with varying maintenance levels	High	v1 validates which tools deliver real value; v2 internalizes critical primitives. Maintain fallback paths for each tier.
Cross-repo scaling: indexing hundreds of repos with multiple tools may exceed practical time/compute limits	Medium	Progressive indexing: IaC first (fast, high confidence), then prioritized service repos. Parallelize tool execution.
Tag store coherence: concurrent writes from multiple tools could create conflicting or duplicate tags	Medium	SQLite WAL mode for concurrent reads; serialized writes per session; deduplication on (target_ref, kind, source_tool).
Agent reasoning quality: autonomous skills depend on LLM agent making good sequencing decisions	High	Methodology encoding in skill prompts; guardrails on autonomous operation; human-in-the-loop for high-stakes decisions.
IaC coverage gaps: not all organizations have comprehensive, up-to-date IaC	Medium	Confidence gradient explicitly models this; fallback to code-level dependency tracing with lower confidence scores.

10.2 Open Questions

1. **Tag store query language:** Should the arch-tags skill expose a SQL interface directly, or an abstracted query API that maps to SQL? Direct SQL is more powerful but requires the agent to write correct queries.
2. **Multi-tenancy in tag store:** For consulting use, each client engagement is a separate tag store. But should the schema support multiple clients in a single store for aggregate analysis (anonymized patterns across engagements)?
3. **Real-time vs. batch operation:** v1 is session-based (consultant triggers analysis). Should the platform support continuous monitoring where tags are updated as code changes, or is batch assessment sufficient?
4. **Rivière schema coverage:** How much of the flow schema can be reliably auto-populated vs. requiring human architectural judgment? This determines the boundary between automated and consultant-driven work.
5. **Commercial tool inclusion:** Greptile provides highest semantic richness through LLM-in-the-loop indexing but is SaaS-only and expensive. Should Archimedes support commercial tools alongside OSS for clients willing to pay for premium analysis?

6. **Skill distribution:** How are skills distributed and versioned? As git repos? npm/PyPI packages? A skills registry? This affects how consulting methodology evolves and is maintained.

11. Implementation Roadmap

11.1 Phase 1: Foundation (Weeks 1–4)

- Define and implement the tag store SQLite schema with CLI read/write interface
- Build arch-tags atomic skill (tag store CRUD operations for agent use)
- Build arch-structure atomic skill wrapping ast-grep with tag output
- Validate on a known codebase (Delos IoT platform: 367+ repos, 941 devices)

11.2 Phase 2: Semantic Layer (Weeks 5–8)

- Integrate code-chunk for chunking and ColGREP for multi-vector search
- Build arch-search atomic skill with tag production
- Integrate qmd for documentation/knowledge base indexing
- Build arch-docs atomic skill wrapping qmd + rlmgrep
- Build first workflow skill: arch-map-service

11.3 Phase 3: Agentic Layer (Weeks 9–12)

- Integrate osgrep for role classification and call-chain analysis
- Integrate Serena for LSP-based navigation
- Build arch-observe and arch-navigate atomic skills
- Build arch-trace-flow and arch-assess-debt workflow skills
- Integrate Rivière for flow schema output

11.4 Phase 4: Autonomy (Weeks 13–16)

- Build arch-map-infra workflow skill (IaC parsing and topology extraction)
- Build arch-modernize autonomous skill with guardrails
- Build arch-investigate autonomous skill for open-ended exploration
- End-to-end validation: full modernization assessment on Delos codebase
- Document methodology and produce client-facing deliverable templates

Appendix A: Tool Reference Links

Tool	Repository / Source	License
ast-grep	github.com/ast-grep/ast-grep	MIT
Rivière / Éclair	github.com/NTCoding/living-architecture	Apache 2.0

Tool	Repository / Source	License
code-chunk	github.com/supermemoryai/code-chunk	MIT
ColGREP (next-plaid)	github.com/lightonai/next-plaid	Apache 2.0
qmd	github.com/tobi/qmd	MIT
osgrep	github.com/Ryandonofrio3/osgrep	MIT
Serena	github.com/oraios/serena	AGPL-3.0
rlmgrep	github.com/halfprice06/rlmgrep	MIT
ChunkHound	github.com/chunkhound/chunkhound	MIT
mgrep	github.com/mixedbread-ai/mgrep	Apache 2.0
Greptile (commercial)	greptile.com	Commercial SaaS

— End of Document —